



# Programmiertechnik II

Graphen mit Kantengewichten

Ralf Herbrich

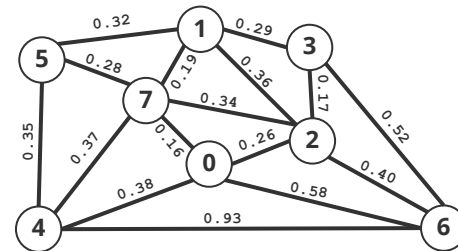
1. Graphen mit Kantengewichten
2. Ungerichtete Graphen: Minimale Spannbäume
  - Prim's Algorithmus
3. Gerichtete Graphen: Kürzeste Pfade
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

1. **Graphen mit Kantengewichten**
2. Ungerichtete Graphen: Minimale Spannbäume
  - Prim's Algorithmus
3. Gerichtete Graphen: Kürzeste Pfade
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

# Graphen mit Kantengewichten

- **Definition (Graph).** Ein **Graph**  $G = (V, E)$  besteht aus einer Menge  $V$  von Knoten (*vertices*, *nodes*) und einer Menge  $E \subseteq V \times V$  von Kanten.
- **Definition (Graph mit Kantengewichten).** Ein Graph  $G = (V, E)$  hat **Kantengewichte** genau dann wenn es eine Funktion  $\omega: E \rightarrow \mathbb{R}$  gibt die jeder Kante  $e \in E$  ein Gewicht  $\omega(e) \in \mathbb{R}$  zuordnet.
- **Kantenrepräsentation:** Für jede Kante wird ein Objekt eines (abstrakten) Datentyps benutzt, der folgende Operationen unterstützt
  - either:  $\rightarrow \{0, 1, \dots, V - 1\}$
  - other:  $\{0, 1, \dots, V - 1\} \rightarrow \{0, 1, \dots, V - 1\}$
  - from:  $\rightarrow \{0, 1, \dots, V - 1\}$
  - to:  $\rightarrow \{0, 1, \dots, V - 1\}$
  - weight:  $\rightarrow \mathbb{R}$

[edge.h](#)



```
// Implements an immutable weighted edge
class Edge {
    int v;
    int w;
    double edge_weight;

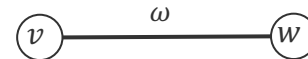
public:
    // returns the weight of the edge
    double weight() const { return (edge_weight); }

    // returns the tail vertex of this edge
    int from() const { return (v); }

    // returns the head vertex of this edge
    int to() const { return (w); }

    // returns either endpoint of this edge
    int either() const { return (v); }

    // returns the endpoint of this edge that
    // is different from the given vertex
    int other(const int vertex) const {
        if (vertex == v) return w;
        else if (vertex == w) return v;
        else throw logic_error("Illegal endpoint");
    }
};
```

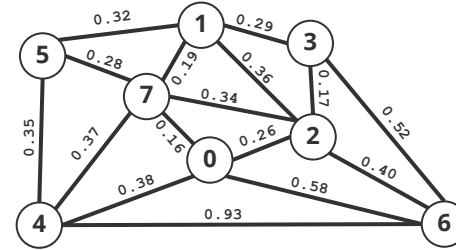


Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Adjazenzlistenrepräsentation

- **Idee:** Für jeden Knoten wird eine verlinkte Liste der Kanten gespeichert, die diesen Knoten enthalten
- In der Praxis benutzt, weil
  1. Algorithmen oft über die Nachbarknoten iterieren
  2. Echte Graphen "dünn" (*sparse*) sind

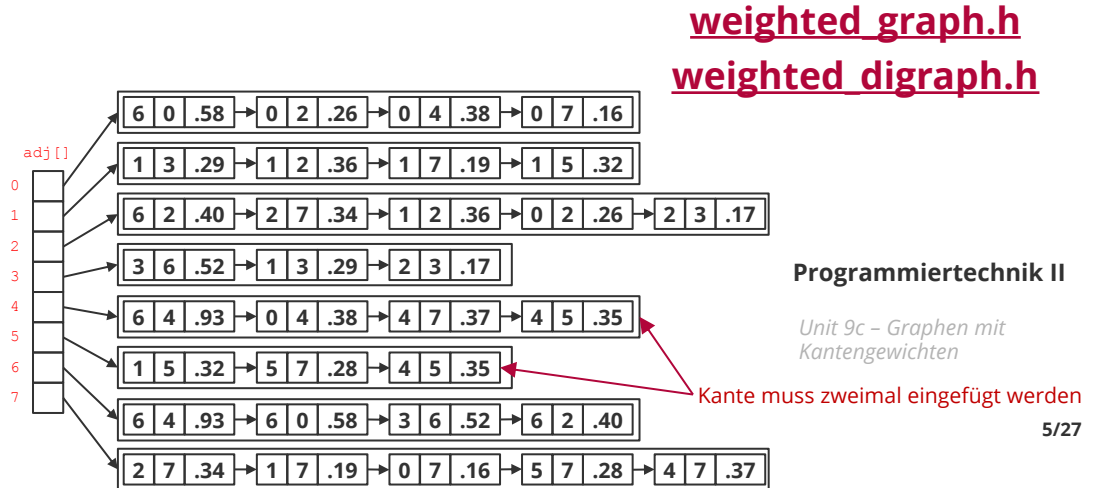


```
// Implements a class representing an undirected edge-weighted graph
class EdgeWeightedGraph {
    int no_vertices; // number of vertices
    int no_edges; // number of edges
    Bag<Edge>* adj_lists; // array of adjacency lists

public:
    // constructor
    EdgeWeightedGraph(const int V : no_vertices(V),
                     no_edges(0)) {
        adj_lists = new Bag<Edge>[V];
    }

    // adds an edge to the undirected graph
    void add_edge(const Edge& e) {
        int v = e.either();
        int w = e.other(v);
        no_edges++;
        adj_lists[v].add(e);
        adj_lists[w].add(e);
        return;
    }

    // return the iterator over the adjacency-list for a vertex
    Bag<Edge>* adj(const int v) const {
        validate_vertex(v);
        return (adj_lists[v]);
    }
};
```

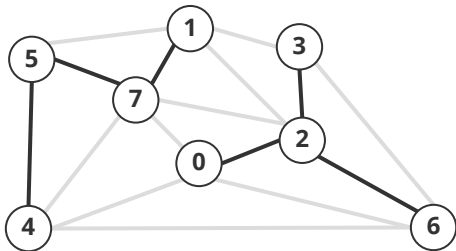
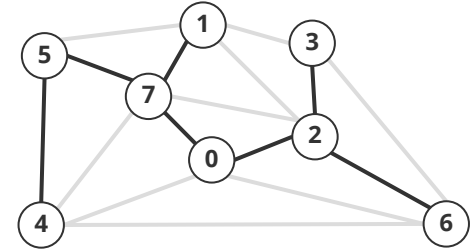


Programmiertechnik II

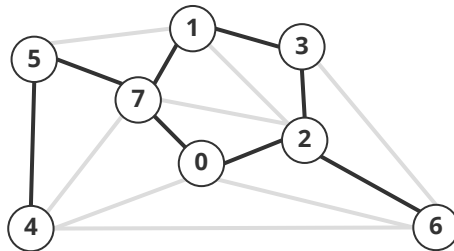
Unit 9c – Graphen mit Kantengewichten

1. Graphen mit Kantengewichten
2. **Ungerichtete Graphen: Minimale Spannbäume**
  - Prim's Algorithmus
3. Gerichtete Graphen: Kürzeste Pfade
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

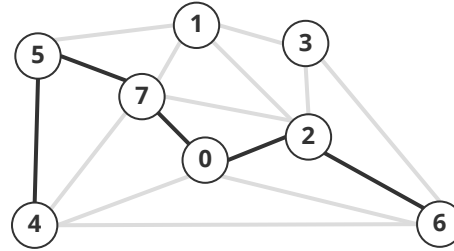
- **Definition (Spannbaum).** Ein **Spannbaum** eines ungerichteten Graphen  $G = (V, E)$  ist ein Teilgraph  $T = (V_t, E_t), V_t \subseteq V, E_t \subseteq E$  der
  - **zusammenhängend** ist
  - **azyklisch** ist,
  - **alle Knoten umfasst**.
- **Folgerung:** Jeder Spannbaum hat genau  $|V| - 1$  Kanten!



Nicht zusammenhängend



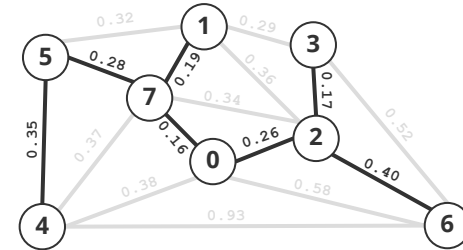
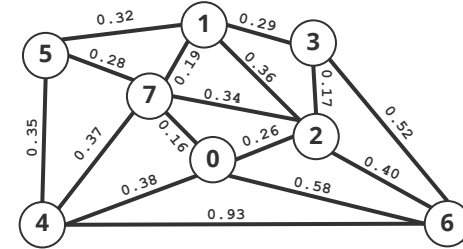
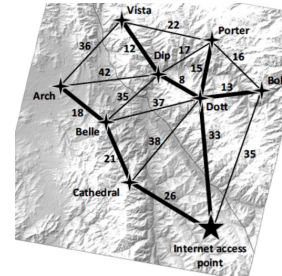
Zyklisch



Umfasst nicht alle Knoten

# Minimaler Spannbaum (*minimal spanning tree (MST)*)

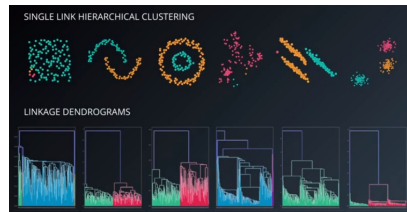
- **Gegeben:** Ein ungerichteter Graph  $G$  mit positiven Kantengewichten
- **Gesucht:** Ein Spannbaum, dessen Summe der Kantengewichte („Kosten“) minimal ist
- **Anwendung von minimalen Spannbäumen:**
  - Netzwerkdesign
  - Konfiguration von Ethernet Bridges
  - Clustering Algorithmen (Künstliche Intelligenz)



**Kosten** =  $0.35 + 0.28 + 0.19 + 0.16 + 0.26 + 0.17 + 0.40 = 1.81$

**Programmiertechnik II**

Unit 9c – Graphen mit Kantengewichten

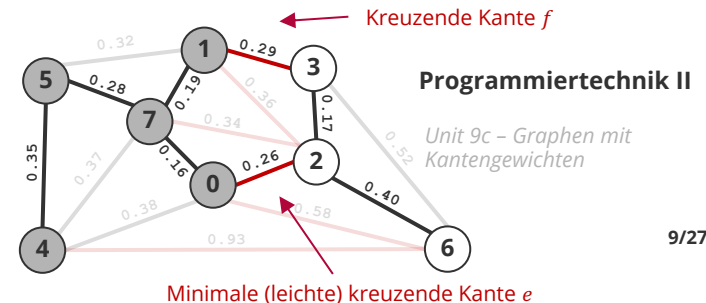
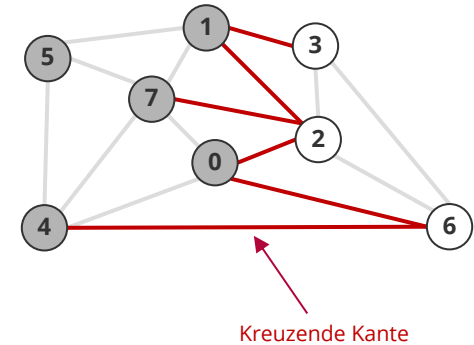


# Schnitteigenschaft (*cut property*)

- **Definition (Schnitt).** Ein **Schnitt** (*cut*) teilt die Knotenmenge  $V$  in einem Graphen  $G = (V, E)$  in zwei nicht leere, disjunkte Teilmengen  $V_1, V_2 \subset V$ , das heißt:  $V_1 \cap V_2 = \emptyset, V_1 \cup V_2 = V, |V_1| > 0, |V_2| > 0$ .
- **Definition (Kreuzende Kante).** Als **kreuzende Kante** (*crossing edge*) in einem Schnitt werden Kanten bezeichnet, die einem Knoten der einen mit einem Knoten der anderen Menge verbinden.
- **Satz (Schnitteigenschaft).** Für jeden Schnitt liegt eine kreuzende Kante mit minimalem Gewicht im minimalen Spannbaum.

□ **Beweis.** Widerspruchsbeweis.

- Angenommen Kante  $e$  wäre nicht im MST sondern Kante  $f$
- Wir entfernen Kante  $f$  und fügen Kante  $e$  in dem MST ein
- Dann sinkt das Gewicht des weiterhin gültigen MST
- Das kann aber per Annahme nicht möglich sein!



1. Graphen mit Kantengewichten
2. Ungerichtete Graphen: Minimale Spannbäume
  - **Prim's Algorithmus**
3. Gerichtete Graphen: Kürzeste Pfade
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

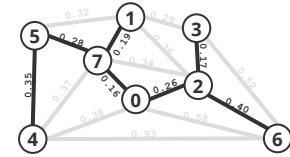
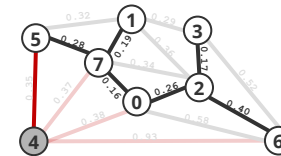
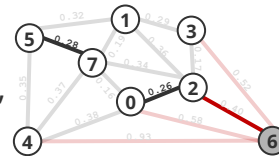
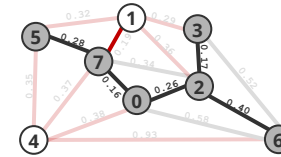
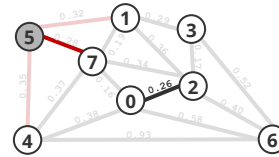
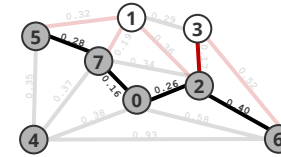
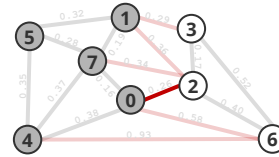
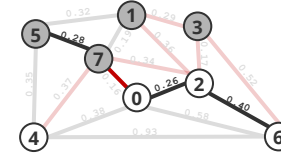
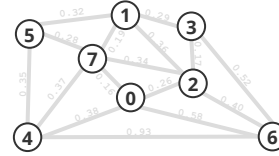
# Greedy Algorithmus

1. Färbe alle Kanten grau
2. Finde einen Schnitt mit keiner schwarzen Kante im Schnitt
3. Färbe die minimale (leichteste) Kante schwarz
4. Wiederhole bis  $|V| - 1$  Kanten schwarz gefärbt sind.

■ **Satz.** Der *Greedy* Algorithmus berechnet den minimalen Spannbaum

■ **Beweis.**

1. Jede schwarze Kante ist per Schnitteigenschaft im minimalen Spannbaum.
2. Solange nicht  $|V| - 1$  Kanten betrachtet wurden, gibt es einen Schnitt ohne schwarze Kanten zwischen den Zusammenhangskomponenten.



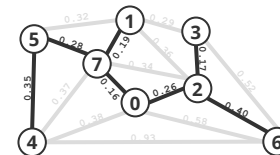
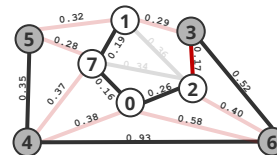
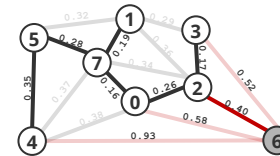
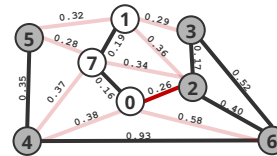
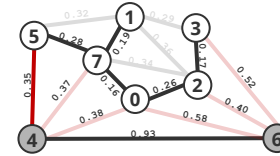
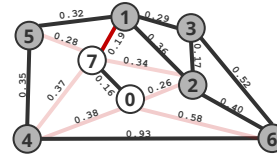
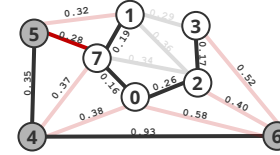
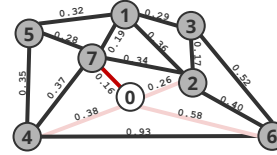
Programmiertechnik II

Unit 9c – Graphen mit Kantengewichten

# Prim's Algorithmus (1957)

- **Idee:** Baue den MST in **einer Zusammenhangskomponente** sukzessiv auf

1. Beginne mit Knoten 0 im MST und füge alle seine Kanten in die Liste der kreuzenden Kanten ein
2. Wähle die Kante in der Liste der kreuzenden Kanten, die minimales Gewicht hat
3. Füge den neuen verbundenen Knoten zum MST und füge alle seine Kanten zur Liste der kreuzenden Kanten hinzu
4. Wiederhole Schritt 2 & 3 bis der MST  $|V| - 1$  Kanten hat oder keine Kanten mehr in der Liste der kreuzenden Kanten sind



Robert Clay Prim  
(1921 – 2009)



Vojtěch Jarník  
(1897 – 1970)



Edsger Dijkstra  
(1930 – 2002)

Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Prim's Algorithmus

## ■ Implementierung:

- Liste der Knoten im MST: boolean Array
- Liste der Kanten im MST:  
Queue<Edge> Warteschlange
- Lister der kreuzenden Kanten:  
MinPQ<Edge> Prioritätswarteschlange

## ■ Aufwand:

- Speicher:  $O(|E|)$
- Zeit:  $O(|E| \cdot \log_2(|E|))$

## ■ Details:

- *Lazy* Prim: Entferne zyklische Kanten nicht während des Algorithmus
- *Eager* Prim: Entferne zyklische Kanten während des Algorithmus und Update aller minimalen Distanzen

```
// Implements a class that computes a minimum spanning forest using a lazy version of Prim's
class LazyPrimMST {
    double mst_weight; // total weight of MST
    Queue<Edge> mst; // edges in the MST

    // add all edges e incident to v onto pq if the other endpoint has not yet been scanned
    void scan(const EdgeWeightedGraph& g, const int v, bool* marked, MinPQ<Edge>& pq) {
        marked[v] = true;
        for (auto& e : g.adj(v)) {
            if (!marked[e.other(v)]) pq.insert(e);
        }
    }

    // run Prim's algorithm
    void prim(const EdgeWeightedGraph& g, const int s, bool* marked, MinPQ<Edge>& pq) {
        scan(g, s, marked, pq);
        while (!pq.is_empty()) {
            Edge e = pq.del_min(); // better to stop when mst has V-1 edges
            int v = e.either(), w = e.other(v); // smallest edge on pq
            if (marked[v] && marked[w]) continue; // two endpoints
            mst.enqueue(e); // lazy, both v and w already scanned
            mst_weight += e.weight(); // add e to MST
            if (!marked[v]) scan(g, v, marked, pq); // v becomes part of tree
            if (!marked[w]) scan(g, w, marked, pq); // w becomes part of tree
        }
    }

public:
    // constructor
    LazyPrimMST(const EdgeWeightedGraph& g) : mst_weight(0) {
        // allocate the marked array
        bool* marked = new bool[g.V()]; // marked[v] = v is on the MST
        for (auto v = 0; v < g.V(); v++) marked[v] = false;

        // run Prim from all vertices to
        MinPQ<Edge> pq; // edges with one endpoint in tree
        for (auto v = 0; v < g.V(); v++)
            if (!marked[v]) prim(g, v, marked, pq);

        // release the marked array
        delete[] marked;
    }
};
```

[prim.h](#)

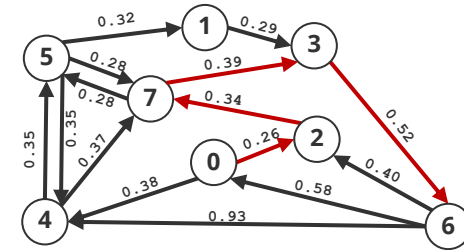
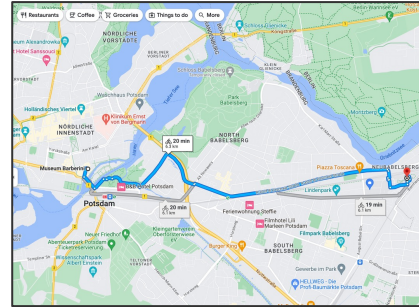
Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

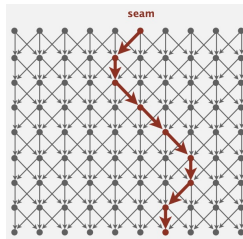
1. Graphen mit Kantengewichten
2. Ungerichtete Graphen: Minimale Spannbäume
  - Prim's Algorithmus
3. **Gerichtete Graphen: Kürzeste Pfade**
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

# Kürzeste Pfade in einem kantengewichteten Digraphen

- **Problem:** Gegeben ein gerichteter Graph  $G = (V, E)$  und zwei Knoten  $s \in V$  und  $t \in V$ , finde einen gerichteten Pfad von  $s$  zu  $t$  so dass kein anderer Pfad von  $s$  zu  $t$  ein geringeres Gewicht hat.
- **Anwendungen von kürzesten Pfaden:**
  - Routenplanung (DHL, Amazon)
  - Bildverarbeitung (*seam carving*)
  - Arbitragen auf Wechselkursen
  - Netzwerkrouting



Kürzester Pfad von 0 nach 6



## Programmiertechnik II

Unit 9c – Graphen mit Kantengewichten

# Kürzeste-Pfade-Baum (*shortest-path-tree (SPT)*)

- **Definition (Kürzeste-Pfade-Baum):** Gegeben ein gerichteter Graph  $G = (V, E)$  und ein Startknoten  $s \in V$ , finde einen azyklischen Teilgraph mit allen Knoten und  $|V| - 1$  Kanten der  $s$  als Wurzel enthält und alle von  $s$  erreichbaren Knoten, so dass jeder Pfad der kürzeste Pfad von  $s$  in  $G$  ist.
  - Verallgemeinerung von MST auf kantengewichtet gerichtete Graphen
  - Darstellung durch zwei Felder

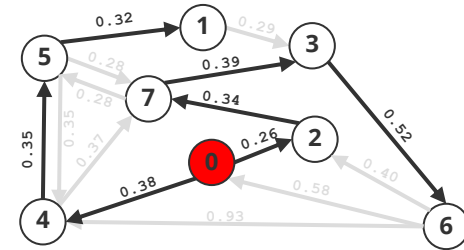
```
// Implements a shortest-path-tree class from s
class SPT {
    int no_vertices; // number of vertices
    double* dist_to; // dist_to[v] = distance of shortest s->v path
    Edge* edge_to; // edge_to[v] = last edge on shortest s->v path

    const Edge null_edge;
};
```

```
// returns a shortest path from the source vertex to vertex v
const Stack<Edge> path_to(const int v) const {
    Stack<Edge> path;

    for (auto e = edge_to[v]; e != null_edge; e = edge_to[e.from()]) {
        path.push(e);
    }
    return (path);
}
```

| v | edge_to[] | dist_to |
|---|-----------|---------|
| 0 | -         | 0       |
| 1 | 5 → 1     | 1.05    |
| 2 | 0 → 2     | 0.26    |
| 3 | 7 → 3     | 0.99    |
| 4 | 0 → 4     | 0.38    |
| 5 | 4 → 5     | 0.73    |
| 6 | 3 → 6     | 1.51    |
| 7 | 2 → 7     | 0.60    |



Kürzeste-Pfade-Baum von 0

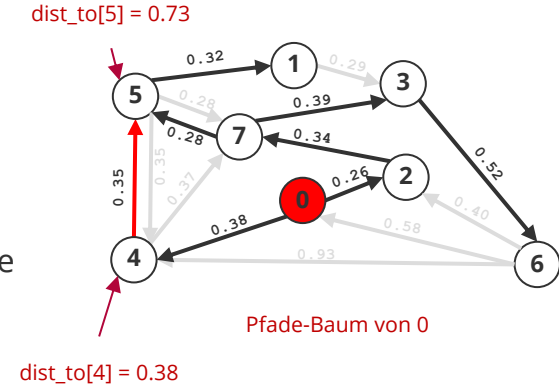
## Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Kantenrelaxation (*edge relaxation*)

- **Idee:** Betrachte die Kante  $v \rightarrow w$  (in Mengenschreibweise:  $(v, w) \in E$ )
  - `dist_to[v]` ist die Länge des kürzesten bekannten Pfads von  $s$  zu  $v$
  - `dist_to[w]` ist die Länge des kürzesten bekannten Pfads von  $s$  zu  $w$
  - `edge_to[w]` ist die letzte Kante im kürzesten bekannten Pfad von  $s$  zu  $w$
  - Wenn der Pfad von  $s$  zu  $w$  durch  $v$  kürzer ist, als der momentane Pfad, passe `dist_to[w]` und `edge_to[w]` an

```
// relax edge e
void relax(const Edge& e) {
    const int v = e.from();
    const int w = e.to();
    if (dist_to[w] > dist_to[v] + e.weight()) {
        dist_to[w] = dist_to[v] + e.weight();
        edge_to[w] = e;
    }
}
```



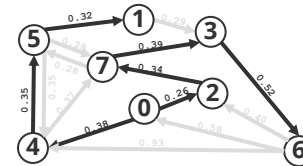
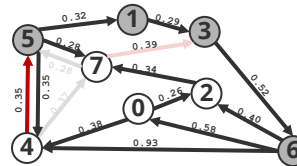
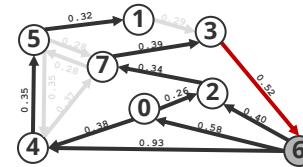
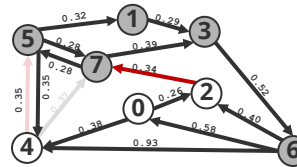
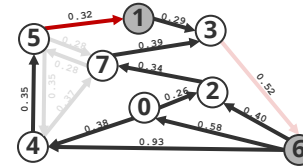
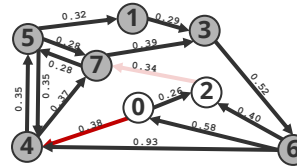
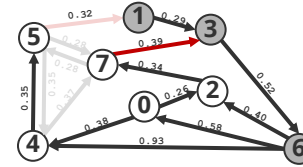
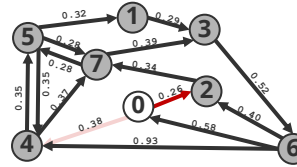
## Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

1. Graphen mit Kantengewichten
2. Ungerichtete Graphen: Minimale Spannbäume
  - Prim's Algorithmus
3. Gerichtete Graphen: Kürzeste Pfade
  - **Positive Kantengewichte: Dijkstra's Algorithmus**
  - Beliebige Kantengewichte: Bellmann-Ford Algorithmus

# Dijkstra's Algorithmus

- **Idee:** Baue den SPT in sukzessiv über die Knoten auf, die am nächsten am Startknoten sind
- 1. Initialisiere alle Distanzen mit  $\infty$  (außer dem Startknoten mit Distanz 0) und färbe alle Knoten grau
- 2. Beginne mit Knoten  $s$  im SPT
- 3. Färbe den momentanen Knotens weiß und relaxiere alle Nachbarkanten
- 4. Wähle die Kante, die zum grauen Knoten  $w$  zeigt, der am nächsten an  $s$  ist
- 5. Wiederhole Schritt 3 & 4 bis alle Knoten weiß gefärbt sind.



Edsger Dijkstra  
(1930 – 2002)

Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Dijkstra's Algorithmus Implementierung

## ■ Implementierung:

- Prioritätswarteschlange über Kanten
- Prioritäten werden separat in einem Schlüssel gesetzt und müssen auch angepasst werden können
- Komplexität:  $|E| \cdot \log_2(|V|)$

## ■ Korrektheitsbeweis:

- Jede Kante  $e = (v, w) \in E$  wird genau einmal relaxed so dass
 
$$\text{dist\_to}[w] \leq \text{dist\_to}[v] + e.\text{weight}()$$
- Diese Ungleichung gilt bis zur Termination
  - $\text{dist\_to}[w]$  kann nicht wachsen
  - $\text{dist\_to}[v]$  kann nicht wachsen

```
// Implements a class that computes the shortest path tree using Dijkstra's algorithm
class DijkstraSP {
    int no_vertices; // number of vertices
    double* dist_to; // dist_to[v] = distance of shortest s->v path
    Edge* edge_to; // edge_to[v] = last edge on shortest s->v path
    const Edge null_edge;

    // relax edge e and update pq if changed
    void relax(const Edge& e, IndexMinPQ<double>& pq) {
        const int v = e.from();
        const int w = e.to();
        if (dist_to[w] > dist_to[v] + e.weight()) {
            dist_to[w] = dist_to[v] + e.weight();
            edge_to[w] = e;
            if (pq.contains(w)) pq.decrease_key(w, dist_to[w]);
            else pq.insert(w, dist_to[w]);
        }
    }

public:
    // constructor
    DijkstraSP(const EdgeWeightedDigraph& dg, const int s) : no_vertices(dg.V()) {
        // initialization of dist_to array
        for (auto v = 0; v < no_vertices; v++) {
            dist_to[v] = numeric_limits<double>::infinity();
            edge_to[v] = null_edge;
        }
        dist_to[s] = 0.0;

        // relax vertices in order of distance from s
        IndexMinPQ<double> pq(no_vertices); // priority queue of vertices
        pq.insert(s, dist_to[s]);
        while (!pq.is_empty()) {
            int v = pq.del_min();
            for (auto& e : dg.adj(v))
                relax(e, pq);
        }
    }
};
```

[dijkstra.h](#)

Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Dijkstra's & Prim's Algorithmus

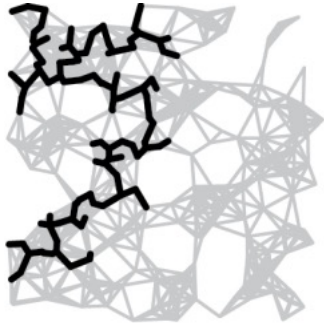
- **Frage:** Sind Dijkstra's und Prim's Algorithmus ähnlich?
  - Es sind essentiell die gleichen Algorithmen!
  - Beide Algorithmen berechnen einen minimalen Spannbaum

Ja

Nein

- **Unterschiede:**

**Prim:** Nächsten Knoten zum minimalen Spannbaum



**Dijkstra:** Nächsten Knoten zum Start



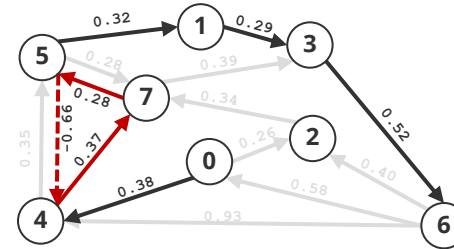
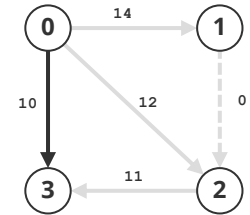
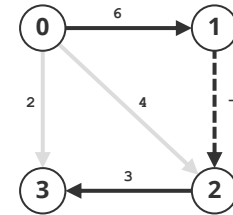
Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

1. Graphen mit Kantengewichten
2. Ungerichtete Graphen: Minimale Spannbäume
  - Prim's Algorithmus
3. Gerichtete Graphen: Kürzeste Pfade
  - Positive Kantengewichte: Dijkstra's Algorithmus
  - **Beliebige Kantengewichte: Bellmann-Ford Algorithmus**

# Negative Kantengewichte und Zyklen

- Dijkstra's Algorithmus funktioniert nicht mehr
  - Neugewichten durch Addition funktioniert nicht!
- **Definition (Negativer Zyklus).** Ein **negativer Zyklus** ist ein gerichteter Zyklus, dessen Summe der Gewichte der Kanten negativ ist
- **Satz.** Ein Kürzester-Pfade-Baum (SPT) existiert nur genau dann wenn es keine negative Zyklen gibt.



Negativer Zyklus  $(-0.66 + 0.28 + 0.37 = -0.01)$

Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Bellmann-Ford Algorithmus

## ■ Grundidee:

1. Relaxiere alle  $|E|$  Kanten in beliebiger Reihenfolge
2. Wiederhole Schritt 1. genau  $|V|$  mal

- ## ■ Man kann zeigen, dass dieses Grundschemata den SPT für einen beliebigen kantengewichteten Graphen ohne negative Zyklen findet (Theoretische Informatik I)

## ■ Optimierung:

- Wenn sich  $\text{dist\_to}[v]$  während des Durchlaufs  $i$  nicht ändert, brauchen wir keine ausgehende Kante von  $v$  in Durchlauf  $i + 1$  relaxen
- **Idee:** Eine Warteschlange von Knoten  $v$  verwalten, deren  $\text{dist\_to}[v]$  sich verändert hat.

```
// Implements a class that computes the shortest path tree in edge-weighted digraph G from vertex s
class BellmanFordSP {
    int no_vertices; // number of vertices
    double* dist_to; // dist_to[v] = distance of shortest s->v path
    Edge* edge_to; // edge_to[v] = last edge on shortest s->v path
    const Edge null_edge;

    // relax vertex v and put other endpoints on queue if changed
    void relax(const EdgeWeightedDigraph& dg, const int v, Queue<int>& queue, bool* on_queue) {
        for (auto& e : dg.adj(v)) {
            int w = e.to();
            if (dist_to[w] > dist_to[v] + e.weight() + numeric_limits<double>::epsilon()) {
                dist_to[w] = dist_to[v] + e.weight();
                edge_to[w] = e;
                if (!on_queue[w]) {
                    queue.enqueue(w);
                    on_queue[w] = true;
                }
            }
        }
    }

public:
    // constructor
    BellmanFordSP(const EdgeWeightedDigraph& dg, const int s) : no_vertices(dg.V()) {
        // allocate memory for dist_to, edge_to, and on_queue array
        dist_to = new double[no_vertices];
        edge_to = new Edge[no_vertices];
        bool* on_queue = new bool[no_vertices];

        // initialization of dist_to array
        for (auto v = 0; v < no_vertices; v++) {
            dist_to[v] = numeric_limits<double>::infinity();
            edge_to[v] = null_edge;
            on_queue[v] = false;
        }
        dist_to[s] = 0.0;

        // Bellman-Ford algorithm
        Queue<int> queue;
        queue.enqueue(s);
        on_queue[s] = true;
        while (!queue.is_empty()) {
            int v = queue.dequeue();
            on_queue[v] = false;
            relax(dg, v, queue, on_queue);
        }

        // free the memory for the on_queue array
        delete[] on_queue;
    }
}
```



Richard E. Bellman  
(1920 – 1984)



Lester Randolph Ford  
(1886 – 1967)

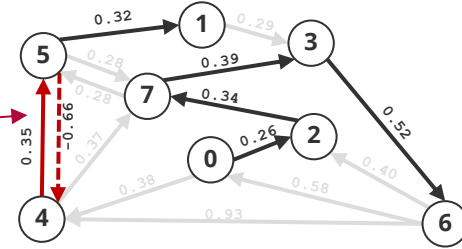
[bellman\\_ford.h](#)

Programmiertechnik II

Unit 9c – Graphen mit  
Kantengewichten

# Bellman-Ford: Negative Zyklen

- **Beobachtung:** Wenn ein Graph einen negativen Zyklus enthält, dann wird der Algorithmus in dieser Schleife hängenbleiben!
- **Idee:** Überprüfe in regelmäßigen Abständen, oder der SPT als Teilgraph einen Zyklus hat (einfache Tiefensuche!)
- **Aufwand** von Zyklenfindung:  $O(|V|)$



```
// relax vertex v and put other endpoints on queue if changed
void relax(const EdgeWeightedDigraph& dg, const int v, Queue<int>& queue, bool& on_queue, int& cost) {
    for (auto& e : dg.adj(v)) {
        int w = e.to();
        if (dist_to[w] > dist_to[v] + e.weight()) {
            dist_to[w] = dist_to[v] + e.weight();
            edge_to[w] = e;
            if (!on_queue[w]) {
                queue.enqueue(w);
                on_queue[w] = true;
            }
        }
    }
    if (++cost % dg.V() == 0) {
        find_negative_cycle();
        if (has_negative_cycle()) return; // found a negative cycle
    }
}

// by finding a cycle in predecessor graph
void find_negative_cycle() {
    EdgeWeightedDigraph spt(no_vertices);
    for (auto v = 0; v < no_vertices; v++)
        if (edge_to[v] != null_edge)
            spt.add_edge(edge_to[v]);

    EdgeWeightedDirectedCycle finder(spt);
    cycle = (finder.has_cycle()) ? new Stack<Edge>(finder.cycle()) : nullptr;

    return;
}
```

[bellman ford.h](http://bellmanford.hpi.uni-potsdam.de/)

## ■ Graphen mit Kantengewichten

- Graphen mit Kantengewichten tauchen in der Praxis häufig auf, auch als Abstraktion für Suchalgorithmen (Künstliche Intelligenz)
- Darstellung über Adjazenslisten am effizientesten

## ■ Ungerichtete Graphen: Minimale Spannbäume & Prim's Algorithmus

- Minimale Spannbäume können über greedy Algorithmen berechnet werden
- Prim's Algorithmus kann effizient mit Prioritätswarteschlangen implementiert werden
- *Eager* Prim ist in der Praxis effizienter als *lazy* Prim

## ■ Gerichtete Graphen: Dijkstra's & Bellmann-Ford Algorithmus

- Kürzeste Pfad können effizient in einem Kürzeste-Pfade-Baum gespeichert werden
- Kürzeste-Pfade-Baum und Minimale Spannbäume sind sehr ähnlich
- Prim's und Dijkstra's Algorithmus sind fast äquivalent!
- Bellmann-Ford mit Warteschlangen ist generischer und fast so effizient wie Dijkstra.

### Programmiertechnik II

*Unit 9c – Graphen mit  
Kantengewichten*

Viel Spaß bis zur nächsten Vorlesung!